University of Waterloo

Faculty of Engineering

Department of Electrical and Computer Engineering

Final Report

StaminaSense: Effortless Health Monitoring for Athletes

Group 2026.013

Jinha Kim (20936456)

Kordian Jonathan Mazurkiewicz (20933336)

Michael Tham (20945854)

Eric Chanthalima (20960497)

Consultant: Maran Ma

30 March 2026

Abstract

Over the past decade, the number of sports games played in a calendar year has drastically increased, leading to shorter recovery times and increased injury risks among athletes. Wearable technologies are used to help trainers manage workload, improve performance, and prevent injury. Current solutions aim to measure performance through distance, exercise times and heat map metrics, while StaminaSense provides detailed overviews of an athlete's physiological state through biometric sensors and motion tracking. This data allows coaches to make educated choices on training, substitutions, and recovery strategies, which can reduce injury and optimize athlete output. StaminaSense uses wearable technology with sensors to obtain data. The system collects input during athletic activity and processes data through a model trained on the athlete wearing the equipment. The output is presented on a mobile device, recommending rest periods when certain thresholds are met. The biggest advantage of StaminaSense over current solutions is its integration of internal physiological monitoring combined with machine learning to offer a simplified approach to athlete management.

Acknowledgements

We would like to extend our sincerest gratitude to our supervisor, Maran Ma, for her guidance, expertise, and support throughout the duration of this capstone project. We are also grateful to the faculty of Electrical and Computer Engineering at the University of Waterloo and the ECE498 instructional team for providing us with the resources and facilities that allowed for the development and research of this project. We would like to recognize the collaborative effort and shared dedication of our entire team. Everyone brought unique contributions to the table, enabling us to realize this project.

Table of Contents

1 High-Level Project Description 4

1.1 Motivation 4

1.2 Project Objective 4

1.3 Block Diagram 4

2 Project Specifications 5

2.1 Functional Specifications 5

2.2 Non-Functional Specifications 6

3 Detailed Design 7

3.1 Software 7

3.1.1 Relational Database 7

3.1.2 Machine Learning Model 8

3.1.3 Backend Server 10

3.1.4 StaminaSense Mobile Application 12

3.2 Wearable Device 14

3.2.1 Microcontroller 14

3.2.2 Hardware/Sensors 20

3.2.3 Power Supply 24

3.3 External Inputs/Outputs 26

4 Discussion and Conclusions 27

4.1 Evaluation of Final Design 27

4.2 Use of Advanced Knowledge 27

4.3 Creativity, Novelty, Elegance 28

4.4 Quality of Risk Assessment 28

4.5 Student Workload 29

5 References 30

List of Figures

Figure 1.1 Block Diagram 4
Figure 3.1 Entity Relationship Diagram 7
Figure 3.2 Machine Learning Flow Diagram 8
Figure 3.3 Machine Learning Flow Diagram 9
Figure 3.4 API Endpoints 11
Figure 3.5 Device Assignment 13
Figure 3.6 Individual Athlete Dashboard 14
Figure 3.7 ESP-WROOM-32 Devkit Pin Mapping [12] 15
Figure 3.8 Microcontroller Workflow 16
Figure 3.9 Microcontroller Pseudo Code 16
Figure 3.10 ECG Wave [15] 17
Figure 3.11 Motion Artifact Synthetic ECG: ground truth vs detected peaks graph 18
Figure 3.12 3D-Printed Enclosure 21
Figure 3.13 Wearable Prototype 21
Figure 3.14 Electrode Configuration 22
Figure 3.15 AD8232 Functional Block Diagram 22
Figure 3.16 Full Schematic with ESP32 without Voltage Regulation on the MCU 24
Figure 3.17 Reduced Schematic with ESP32 25

List of Tables

Table 2.1 Functional Specifications of StaminaSense 5
Table 2.2 Non-Functional Specifications of StaminaSense 6
Table 3.1 Final Model Performance Comparison 10
Table 3.2 Backend Server Weighted Decision Matrix 11
Table 3.3 Pin Mappings 15
Table 3.4 Microcontroller Decision Matrix 19
Table 3.5 Functional Block Diagram: Values for HPF come from the eval board [18] 23
Table 3.6 Power Supply Decision Matrix 26
Table 4.1 Student Workload 29

1 High-Level Project Description

1.1 Motivation

As the demand to generate revenue from sports grows, the sporting calendar continues to expand, resulting in shorter recovery periods and an increased risk of injury for athletes. In professional soccer, this trend is evident in knockout tournaments as organizers expand formats to include more teams. For example, UEFA Champions League, Europe's continental club competition, recently revamped its format, increasing the total amount of matches by 64 [1]. Additionally, in the 2023-24 season, top men's football clubs in Europe's major leagues incurred €732 million in wage losses due to player injuries, which highlights the significant financial and physical toll of injuries [2]. Wearable technologies have been developed to measure performance through distance, exercise times and heat map metrics. Catapult's GPS tracker is one example of such that is used by elite soccer teams such as Bayern Munich, Chelsea, and Borussia Dortmund [3]. There is a notable absence of tools that leverage an athlete's biometric state to accurately assess fatigue.

1.2 Project Objective

The objective of this project is to provide comprehensive overviews of an athlete's physiological state using a wearable vest with machine learning algorithms. The compression vest collects real-time data through embedded biometric sensors. Based on this data, the machine learning model outputs a stamina level. This enables coaches to make more educated choices on training and recovery strategies, ultimately reducing injuries and optimizing athlete output.

1.3 Block Diagram

The system architecture of the proposed solution in shown below in Figure 1.1.

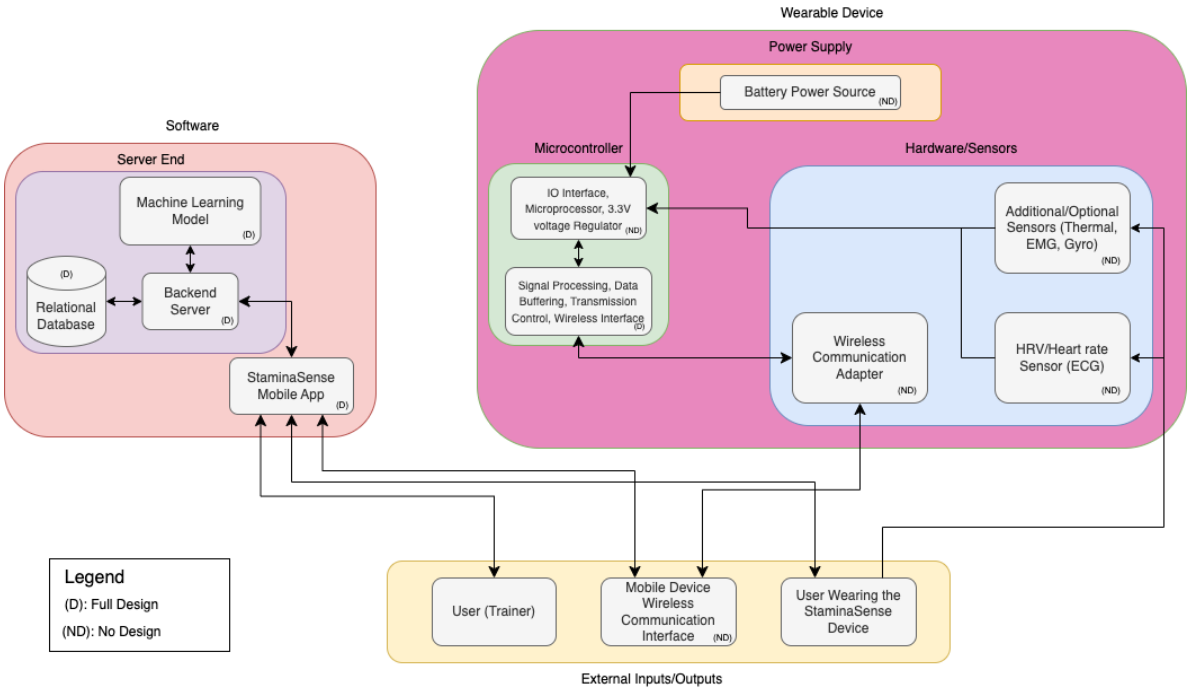

Figure 1.1 Block Diagram

2 Project Specifications

2.1 Functional Specifications

Table 2.1 below outlines the essential and non-essential functional specifications of the project.

Table 2.1 Functional Specifications of StaminaSense

Subsystem	*Specification*	*Necessity*	*Description*
iOS Application	Visualize data for all athletes	Essential	The application displays fatigue levels and biometric data for all athletes being trained.
	Wearable Device Assignment	Essential	The app allows the coach to assign each wearable device to a specific athlete within their training session.
Data Storage	Athlete Profile	Essential	Stored athlete information includes information, age, height, weight, and resting biometric data.
	Coach Profile	Essential	Stored coach information includes login information and the athletes registered in their training session.
User Initialization	Initialization Stage	Essential	The system guides the user through an onboarding session, including introducing device assignment and collecting baseline biometric data.
Microcontroller	IO (input/output) of microcontroller board	Essential	It has enough IO pins to support all sensors and wireless modules.
	Microprocessor	Essential	Able to develop and run a program that manages signal processing, buffer management, and transmission control.
Power Supply	Charging Capabilities	Not Essential	Batteries can charge.
Machine Learning Model	Model Personalization	Essential	The model is personalized to each user through normalizing inputs, as each user's baseline biometrics will differ.

2.2 Non-Functional Specifications

Table 2.2 outlines the essential and non-essential non-functional specifications of the project.

Table 2.2 Non-Functional Specifications of StaminaSense

Subsystem	*Specification*	*Necessity*	*Description*
iOS Application	Data Latency	Not essential	Maximum delay of 10 seconds to get an athlete's most recent statistics and fatigue level.
Machine Learning Model	Performance Targets	Essential	The AI Model reaches these metrics during validation: precision ≥ 80% and cluster accuracy at ≥ 80% alignment.
Wearable Device	Product Weight	Essential	The entire device weighs less than 10 pounds.
	User Experience Constraint	Essential	The device must maintain full sensor functionality after 100 hours of athletic activity, including stretching, sweat exposure, and compression, and must not cause skin irritation or chafing after continuous wear for 8 hours.
Power Supply	Voltage Output	Essential	Provides ~3.3-6V to power the Vin pin. So that the on-board regulator will regulate a constant 3.3V to all pins and power the microprocessor.
	Device Operational Lifetime	Essential	Battery life of > 2 Hrs.
Microcontroller	Transmission Latency Per HRV Snapshot	Essential	< 8 seconds of latency is required between snapshots.
	Wireless Communication Distance	Not Essential	Wireless communication module distance of > 200m from mobile device to module.
Sensor	Sensor Shock Durability	Essential	Mild impact will not affect sensor accuracy by over 15%.
	Sensor Accuracy	Essential	Sensor data >80% accurate while in motion
	Wireless Bandwidth	Essential	> 100 Kbps bandwidth
Backend Server	Latency	Essential	The backend server takes less than 4 seconds per request when handling 20 or less simultaneous user requests.

3 Detailed Design

3.1 Software

3.1.1 Relational Database

The relational database is a Postgres database separated into three separate tables: athletes, coaches, and baseline. See Figure 3.1 for the entity relationship diagram.

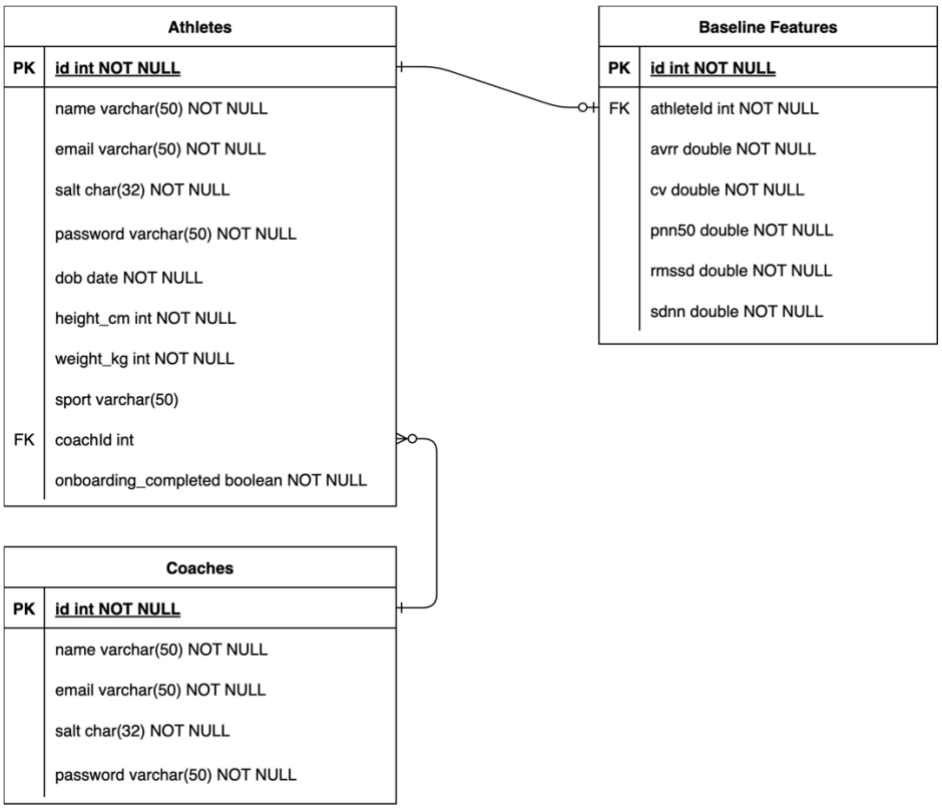

Figure 3.1 Entity Relationship Diagram

The final design of the relational database subsystem consists of a structured database architecture which supports the management of athlete data, measurement of performance metrics, and AI-based fatigue classification.

The athletes table holds static information of an athlete such as login credentials, personal and biological data (e.g., height and weight), and a foreign key to the coach's ID to maintain the relationship between an athlete and their trainer. The coach's table is like the athlete table but omits biological metrics, as these are unnecessary for coaches. This design satisfies FS3 and FS4 from Table 2.1, ensuring clear role-based separation and associations. An *onboarding_complete* field was also included in the athletes table to support athlete initialization, addressing FS5.

The fatigue classifier uses percent change relative to an individual's baseline as its input, rather than raw biometric values. This design accounts for natural physiological differences between individuals, such as varying resting heart rates, ensuring that the model evaluates deviations from a person's normal

state instead of comparing absolute values across different users. This decision will be expanded upon in section 3.1.2. Thus, the design only requires the storage of baseline biometric data rather than all raw data. This approach ensures constant-time updates, constant-time queries, and minimizes storage overhead.

3.1.2 Machine Learning Model

Our machine learning model subsystem is a cloud-hosted Extra Trees model trained on a feature set composed of both static (e.g., age, weight) and dynamic biometric data (e.g., HRV), with a fatigue rating as the target output. The model uses percent change relative to an individual's baseline as its input, rather than raw biometric values. Normalized inputs will be calculated as follows:

$$Normalized\ Input = \frac{(Input - Baseline\ Data)}{Baseline\ Data}$$

Static features like age or sport type, which are not continuously recorded, are included unnormalized. Previous work in heart rate variability research used percent change from the baseline to understand physiological responses. For example, resistance exercise studies compute percent changes in RMSSD to assess autonomic recovery [4]. This work supports the use of percent change as a normalization strategy, enabling meaningful comparisons of individuals with different baseline statistics, helping to personalize our model and satisfy FS9. Figure 3.2 below shows a flow diagram of the process.

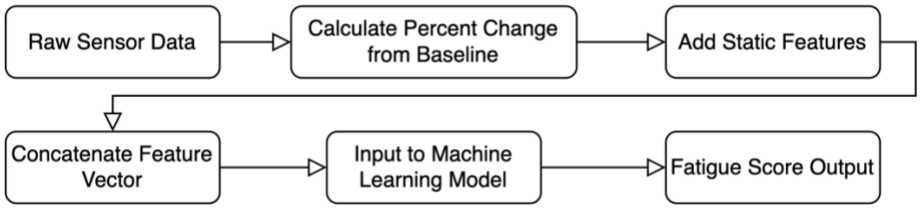

Figure 3.2 Machine Learning Flow Diagram

A critical design decision was how to implement personalization within the machine learning model. Due to user variability in physiology and training routines, a single general model would be insufficient. Two alternatives were considered. The first was to store a local model on each wearable device. This would enable real-time predictions and full personalization, but device limitations in memory, power, and processing would restrict the complexity and scalability of on-device models [5]. The second alternative was to have a global model on the cloud and personalize predictions through the preprocessing of inputs through user-specific data. Although cloud-based predictions require network connectivity and add latency, they allow for larger, more expressive models that can incorporate more predictors. We selected the cloud-based approach as the optimal solution.

Another key decision came with data availability. Currently there are no publicly available datasets that align with our use case, which involves predicting subjective fatigue levels based on biometric data. Therefore, we developed a custom dataset, tailored to our use case. To ensure the model reflects real-world physiology and user experience, we deliberately chose to collect real human data; a process that requires ethics approval from the University of Waterloo. Despite this added effort, we saw it as essential to build a reliable and trustworthy system. Our data collection protocol includes recording static metrics (e.g., height, weight), collecting dynamic data (e.g., HRV) from specific drills, asking users

to report subjective fatigue ratings post-exercise on a 5-point Likert scale, and anonymizing the given data. By starting with ethically sourced, high-quality data, we ensure that personalization and model calibration reflect real-world physiology.

The model underwent four major design iterations, each informed by evaluation feedback and aimed at satisfying the performance targets defined in Table 2.2 NFS 2: precision ≥ 80% and cluster accuracy ≥ 80% alignment. Figure 3.3 traces the 10-fold stratified cross-validation accuracy of five candidate models across these iterations.

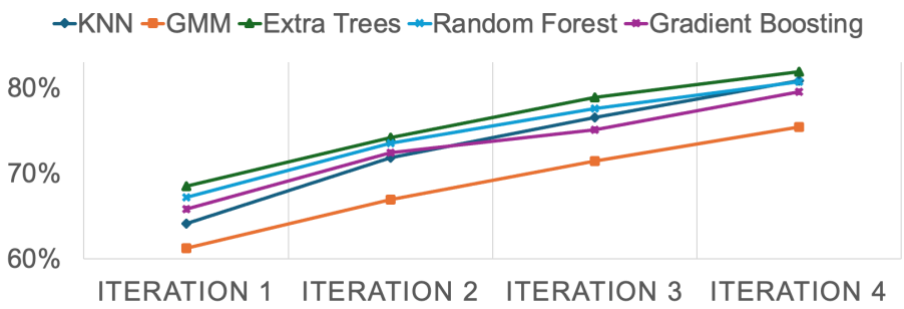

Figure 3.3 Machine Learning Flow Diagram

The first iteration used the original 194-sample dataset collected from 5 athletes. We trained six classifiers (KNN, Gradient Boosting, Extra Trees, Random Forest, SVM, Logistic Regression) alongside a supervised Gaussian Mixture Model. Without feature engineering or baseline normalization, all models performed between approximately 61% (GMM) and 68% (Extra Trees). None satisfied the NFS 2 performance target.

The second iteration refined the preprocessing and introduced per-athlete percent-change normalization. As a result, performance was increased across all models. Extra Trees and Random Forest rose to approximately 74%, while GMM improved to approximately 68%. These preprocessing changes contributed a roughly 6 to 8 percentage point improvement across models, confirming that domain-informed normalization substantially reduces inter-athlete variability in the feature space.

The third iteration involved hyperparameter tuning and evaluation refinement. This involved the adoption of 10-fold stratified cross-validation instead of unreliable single train-test splits. The single train-test splits produced highly variable accuracy estimates, ranging from approximately 40% to 70% depending on the random seed. This variation shows that a single split does not provide a stable estimate of model performance. In contrast, 10-fold stratified cross-validation yields a more reliable evaluation by testing the model across multiple balanced partitions of the dataset which provided more reliable performance observations. Stratification preserves the distribution of fatigue levels across different folds, which is important because the class counts were not perfectly balanced throughout the numerous iterations. In addition to overall accuracy, the project reports weighted precision, weighted recall, and a weighted F1-score. These metrics provide a more complete view of performance and help demonstrate that the model not only performs well on the largest class but maintains reliability across the full fatigue scale. With tuned hyperparameters (300 estimators, balanced class weights), Extra Trees reached approximately 79% and Random Forest approximately 79%. GMM remained at approximately 71%, and KNN reached approximately 76%.

The fourth iteration of models were trained on an expanded dataset. We expanded the dataset from 194 to 1,930+ labeled samples by collecting additional data from multiple volunteers under varied exercise intensities. This was the single largest accuracy driver. Extra Trees reached approximately 82%, Random Forest approximately 81%, and Gradient Boosting approximately 80%, with all three exceeding the 80% NFS 2 performance target. GMM reached approximately 75%, and KNN approximately 81%. Table 3.1 outlines the final performance comparison between all the models tested throughout each iteration also revealing Extra Trees as the best performing model for our project.

Table 3.1 Final Model Performance Comparison

Model	Accuracy	Precision (weighted)	Recall (weighted)	F1-score (weighted)
Random Forest	81.2%	81.0%	81.2%	81.0%
Extra Trees	82.1%	81.9%	82.1%	81.8%
Gradient Boosting	79.8%	79.7%	79.8%	79.5%
KNN (k = 3)	81.0%	80.8%	81.0%	80.7%
GMM	75.2%	74.9%	75.2%	74.8%

Extra Trees was selected as the final deployed model due to its strong performance across the non-functional requirements. It achieved the highest cross-validation accuracy and F1-score amongst candidate models, demonstrating the best generalization capabilities. It satisfied the performance thresholds defined in NFS 2 (Table 2.2), with both precision and accuracy exceeding 80%.

The key advantage of Extra Trees lies in its ability to reduce overfitting through additional randomization. Unlike Random Forest, which searches for optimal split thresholds at each node, Extra Trees selects random thresholds which allows the model to generalize better to unseen patterns and data which is consistent with the nature of live biometric readings. We kept this model in our comparison throughout the design iterations and found that it remained one of the strongest performers as preprocessing and dataset size improved. Extra Trees also aligns well with the fatigue prediction problem. The subsystem combines normalized biometric features with static athlete information, and the relationship between these inputs and reported fatigue is not expected to follow a simple linear pattern. By using an ensemble of randomized decision trees, the model identifies more complex patterns in the feature space of live biometric readings while still producing class predictions that map directly to the 1 to 5 fatigue scale.

3.1.3 Backend Server

The backend server is implemented as a Representational State Transfer (REST) - based Application Programming Interface (API) service using TypeScript and Express.js, designed to handle communication between the mobile application, database, and machine learning model. The server follows a layered architecture pattern with clear separation of concerns between routes, controllers, models, and middleware. The routes layer handles all HTTPs request routing and authentication. The controller layer implements business logic and request handling. The model layer defines data structures

and database interactions. Finally, the middleware layer provides cross-cutting concerns like authentication and error handling.

Table 3.4 Backend Server Weighted Decision Matrix

Criteria	Weight	**Express.js/TS**	**Django/Python**	**Spring/Java**
Type Safety	0.3	9	6	9
Learning Curve	0.2	8	7	5
ML Integration	0.2	7	9	6
Performance	0.15	8	7	9
Community Support	0.15	9	8	8
Total		8.25	7.25	7.45
Rank		1	3	2

The technology stack was decided upon through the decision matrix in Table 3.2. The weights for each criterion were assigned based on our project's priorities. **Type safety** received the highest weight (0.3) because ensuring data integrity and minimizing runtime errors are critical for a health-monitoring application. **Learning curve** and **machine learning (ML) integration** were weighted equally (0.2) to balance the need for rapid development with the core requirement of interfacing with our Python-based machine learning model. Finally, performance and community support (both 0.15) were considered important, but secondary, as all frameworks under consideration offer adequate performance for our load requirements and have robust communities. The technology stack of Express.js with TypeScript was chosen due to the team's comfort and experience with it, strong type safety enhances code reliability and maintainability, it has excellent performance characteristics, and it has a large ecosystem of middleware and libraries.

The authentication system uses JSON Web Tokens (JWT) stored in Hypertext Transport Protocol (HTTP) cookies, implementing the following security measures: token-based session management, secure password hashing using Password-Based Key Derivation Function 1 and 2 (PBKDF2) with a 512-bit Secure Hash Algorithm (SHA-512), and role-based access control for athletes and coaches. This design satisfies FS3 and FS4 by ensuring secure data access and user management.

The application programming interface (API) itself includes create, read, update delete (CRUD) operations which follow a Representational State Transfer (RESTful) design pattern. The API structure and endpoints are outlined in Figure 3.4 below.

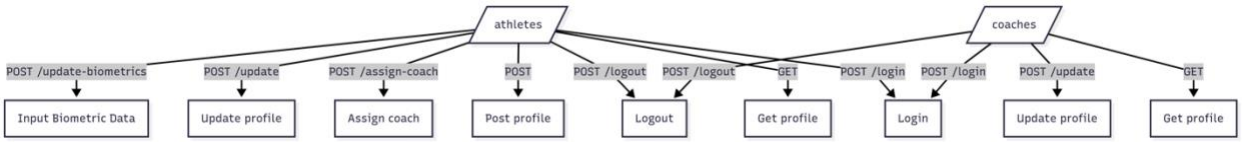

Figure 3.4 API Endpoints

Our latency analysis confirms that the server architecture satisfies the NFS1 and NFS12 requirement of processing requests in under 4 seconds with 20 concurrent users. This performance is achieved through several key optimizations. The use of a persistent database connection pool (5 connections) significantly reduces the overhead of establishing new connections for each request, a time-

intensive operation. Furthermore, the server's asynchronous, non-blocking architecture, inherent to Node.js, allows it to handle I/O-bound tasks like database queries efficiently.

To validate this, simulated load testing was modeled using **k6**, an open-source load testing tool. Under a simulated load of 20 concurrent users making a total of 1,000 requests, the server demonstrated robust performance. The average response times were measured as follows: authentication requests averaged **55ms**, data retrieval requests averaged **70ms**, and data update requests averaged **95ms**. These results are well within the non-functional requirement of under 4 seconds per request, confirming the design's efficiency.

The architecture is explicitly designed for horizontal scalability to accommodate future growth. The use of stateless JWT authentication is central to this design, as it removes the need for session storage on the server. Any server instance can authenticate any request, enabling seamless scaling behind a load balancer without the complexity of synchronized session states. Key API endpoints were also designed for efficiency. For instance, user and coach retrieval operations leverage indexed database lookups, resulting in a time complexity of O(log N), ensuring that performance degrades gracefully as the user base (N) grows.

Finally, a robust error-handling mechanism is central to the server's reliability. The validation middleware proactively catches malformed requests before they reach the business logic, preventing unnecessary processing and potential vulnerabilities. When an error occurs, the server sends a structured JSON response with a clear message, which simplifies debugging for front-end developers and allows the mobile application to provide meaningful feedback to the user, enhancing system stability and overall user experience.

3.1.4 StaminaSense Mobile Application

The StaminaSense Mobile Application serves as the primary user interface for coaches and athletes, allowing them to monitor athlete data, view fatigue predictions, and manage their teams. It is developed as a cross-platform application using React Native with Expo, ensuring it can be deployed to both iOS and Android from a single codebase.

The choice of React Native with Expo was based on a careful evaluation of project requirements, driven by the need for cross-platform compatibility, a streamlined developer experience, and our team's existing proficiency in JavaScript and React. This stack allows us to write code once and deploy it on both iOS and Android, significantly reducing development time and maintenance overhead. The Expo framework further accelerates development by abstracting away native build complexities and enabling over-the-air (OTA) updates.

The application's architecture is designed to be modular and scalable, following a standard component-based structure. This promotes a clear separation of concerns, organized into several layers. Top-level screens represent a full view in the app, which are built from reusable UI components. A centralized navigation layer using React Navigation manages the flow between screens. For application-wide state, such as user authentication and athlete data, a global state management solution using Redux

Toolkit provides a single source of truth. Finally, a dedicated API services layer encapsulates all communication with the backend server, keeping network logic separate from the UI.

The application's core functionality is delivered through several key screens designed to meet the functional specifications, including both an authentication flow and the main application interface. The authentication flow consists of a login screen, where coaches or athletes can enter their email and password to access their accounts via the 'POST /coaches/login' and 'POST /athletes/login' backend endpoints, as well as a registration screen that allows a new user to create an account. Once a user creates an account and is authenticated, the system guides the user through a one-time onboarding session, introducing the device assignment mechanism and collecting baseline biometric data (FS5). Once authenticated, users enter the main application flow (FS1, FS2), which begins with the dashboard screen. This serves as the primary landing page and displays a list of all athletes assigned to the coach. Each entry shows the athlete's name and current real-time predicted fatigue level, retrieved from the 'GET /athletes' endpoint and the 'POST /update-biometrics' endpoint. From the dashboard, selecting an athlete navigates to the athlete detail screen, which provides a comprehensive view of the athlete's biometric data, historical fatigue levels visualized through graphs, and the current predicted fatigue level. Additionally, the application includes a device assignment screen that enables coaches to assign specific wearable devices to athletes, fulfilling the requirements of FS2.

The following figures, Figure 3.5 and 3.6, are the key screens described in this document, including the device assignment page, and the detailed athlete view from the final prototype.

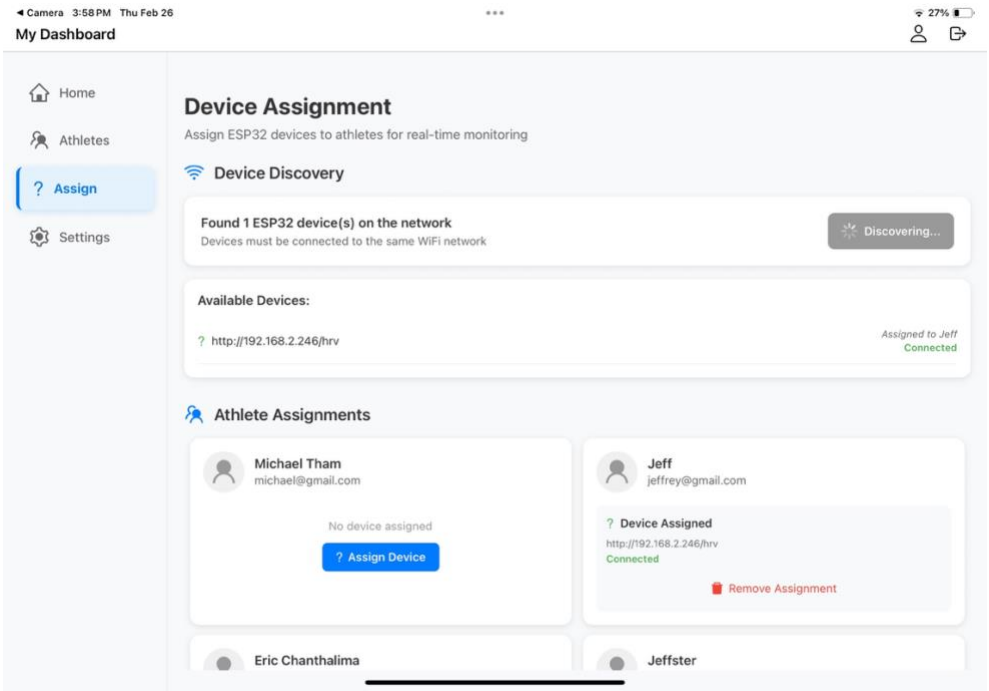

Figure 3.5 Device Assignment

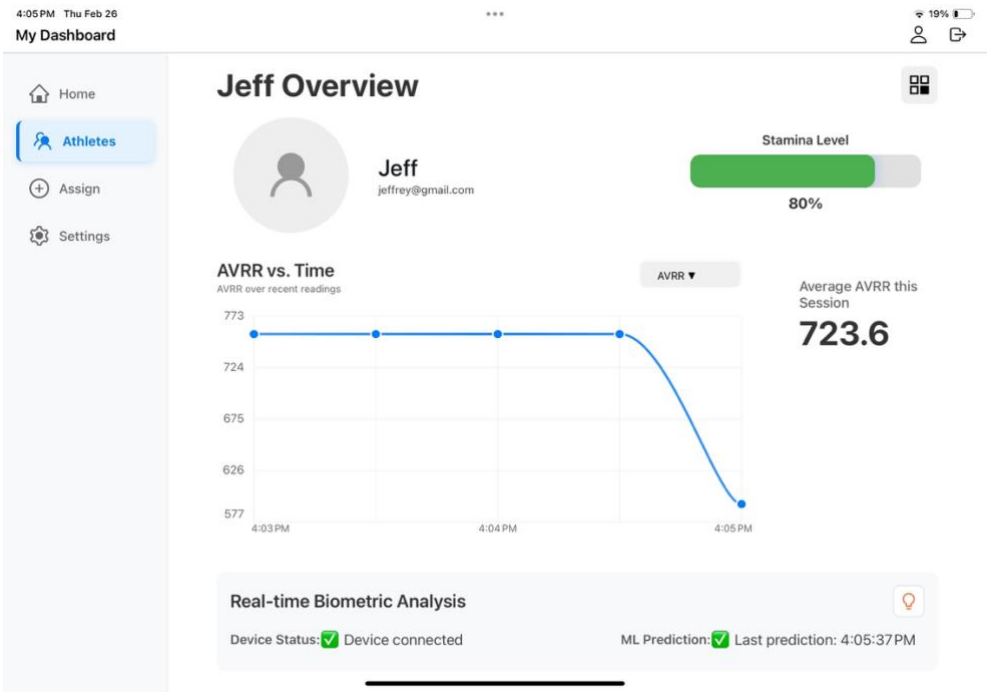

Figure 3.6 Individual Athlete Dashboard

3.2 Wearable Device

3.2.1 Microcontroller

This system's microcontroller is based on the ESP32 chip, developed by Espressif Systems [8], and mounted on an Elegoo development board [12]. This board features a USB Type-C interface and the standard 30-pin configuration common to most ESP32 devkits. The ESP32 is a low-power microcontroller with integrated Wi-Fi and Bluetooth, analog and general-purpose input/output (GPIO) support, and compatibility with multiple integrated development environments (IDEs) for flashing and code development.

In this system, the ESP32 is the central processing unit for electrocardiogram (ECG) signal data. It samples the analog output from the AD8232 development board, preprocesses the signals to detect and calculate heart rate variability (HRV) metrics, and transmits the results over Wi-Fi via HTTP to a web server hosted directly on the ESP32. All signal processing is performed on the microcontroller, and the web interface only displays the collected data. The ESP32 collects 1.5 seconds of ECG data (approximately 375 samples at 250 Hz), computes heart rate variability (HRV) features, and sends the data as a JSON payload to the webserver [8].

To connect components to this board, we are using a prototyping circuit board, which allows the ESP32 dev kit to be securely mounted alongside other modules. subsystems. The pin mapping on the ESP-WROOM-32 Elegoo board is shown below in Figure 3.7 and in Table 3.3.

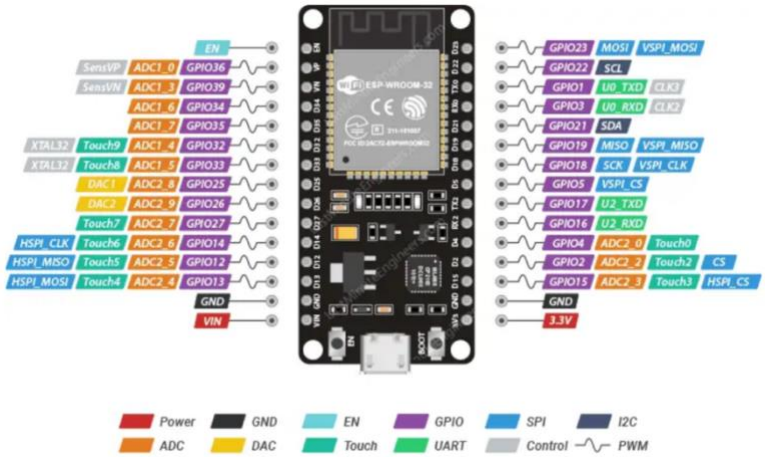

Figure 3.7 ESP-WROOM-32 Devkit Pin Mapping [12]

Table 3.5 Pin Mappings

Connected Component	*Pin Function*	*ESP-WROOM-32 Connection*
Power Supply	Power Input	VIN
Power Supply/Ad8232 GND	Common Ground	GND
Ad8232 Output	Analog ECG Input	GPIO34
Ad8232 LO-	Leads-off Detect -	GPIO33
Ad8232 LO-	Leads-off Detect +	GPIO32
Ad8232 3.3v	Ad8232 3.3v power supply	3.3v

*AD8232 is the ECG heart rate monitor module (Subsystem 3.2.2)

The microcontroller firmware first connects to Wi-Fi and starts the webserver on a specified port. Next, ECG data is continuously from the analog input, and stores samples in a buffer until it is full. Once the buffer is full, HRV features are calculated, and the result is transmitted as a JSON payload. The workflow can be shown below in Figure 3.8.

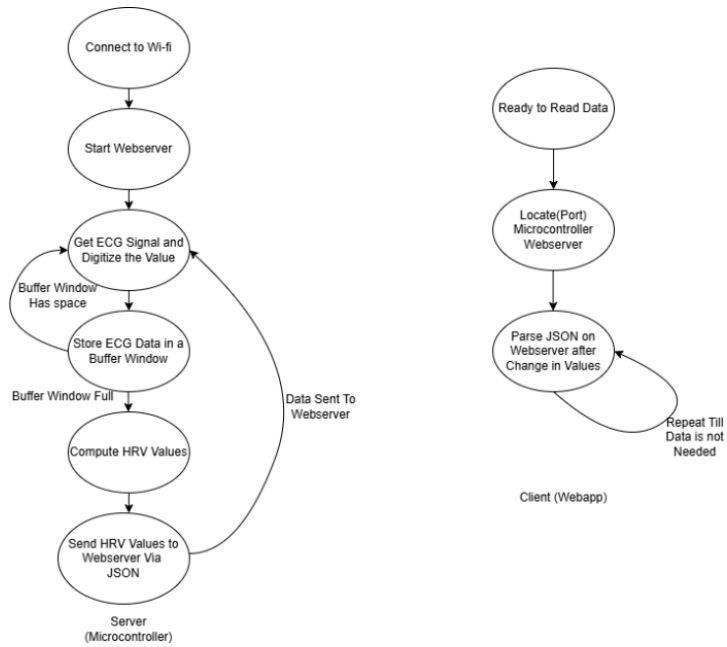

Figure 3.8 Microcontroller Workflow

```
const int fs = 250;                    // Sampling frequency in Hz
const int windowSize = 375;            // 1.5 seconds * 250 Hz
int ecgBuffer[windowSize];
int nnIntervals[MAX_NN];
int index = 0;

setup() {
  connectWiFi();
  StartWebserver();
}

loop() {
  ecgBuffer[index++] = analogRead(ECG_PIN);
  delay(1000 / fs);                    // Sample every 4ms

  if (index >= windowSize) {
    detectRPeaks(ecgBuffer, nnIntervals);

    float avnn = mean(nnIntervals);
    float sdnn = stddev(nnIntervals);
    float rmssd = computeRMSSD(nnIntervals);
    float sdsd = computeSDSD(nnIntervals);
    int nn50 = countNN50(nnIntervals);
    float pnn50 = nn50 / float(length(nnIntervals));
    float hrvIndex = computeHRVIndex(nnIntervals);

    sendToServer(avnn, sdnn, rmssd, sdsd, nn50, pnn50, hrvIndex);
    index = 0;
  }
}
```

Figure 3.9 Microcontroller Pseudo Code

The decision to choose a base sampling frequency of 250 Hz can be shown by quantitatively analyzing the frequency using the **Nyquist Theorem**:

$$f_s \geq 2 * f_{max}$$

Where f_s is the sampling frequency and f_{max} is the expected upper frequency of the ECG signal component. As mentioned in [13], the QRS complex, which is the peak electric signal in the periodic

heart rate is usually contained within 8-50Hz, while abnormal heart rates can be as high as 70Hz. Thus, as shown in Figure 3.9, a sampling frequency of f_s **= 250Hz** satisfies the Nyquist Theorem:

$$250 \geq 2 * 70 \rightarrow 250 \geq 140$$

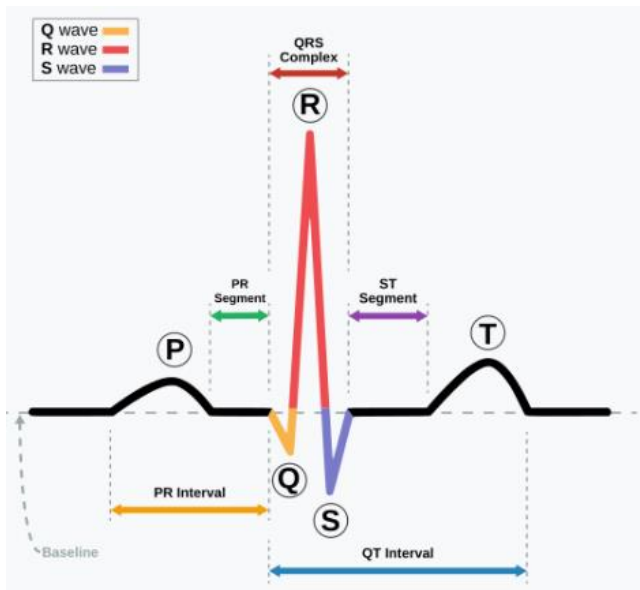

Figure 3.10 ECG Wave [15]

Based on quantitative analysis of the ECG data illustrated above in Figure 3.10, the following equations correspond to the calculations implemented in Figure 3.9, where R_n represents the time of the *n*th R-peak.

*Step 1: Define **NN** intervals*

$$NN_n = R_{n+1} - R_n$$

Step 2: AVNN (Average of NN intervals)

$$\textbf{AVNN} = \frac{1}{n}\sum_{i=1}^{n} NN_i$$

Step 3: SDNN (Standard Deviation of NN intervals)

$$\textbf{SDNN} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(NN_i - AVNN)^2}$$

Step 4: RMSSD (Root Mean Square of Successive Differences)

$$\Delta_n = NN_{n+1} - NN_n$$

$$\textbf{RMSSD} = \sqrt{\frac{1}{2}(\Delta_n^2 + \Delta_{n+1}^2)}$$

Step 5: SDSD (Standard Deviation of Successive Differences)

$$\textbf{SDSD} = \sqrt{\frac{1}{2} \times [(\Delta_1 - \Delta^-)^2 + (\Delta_2 - \Delta^-)^2]}$$

where $\Delta^- = \frac{(\Delta_{n+1} + \Delta_n)}{2}$

Step 6: NN50 and pNN50

$$\textbf{NN50} = Number\ of\ |\Delta_i| > 50\ ms$$

$$\textbf{pNN50} = \frac{NN50}{Total\ NN\ Intervals}$$

$$\textbf{pNN50} = \frac{Number\ of\ |\Delta_i| > 50}{n}$$

By the nature of the use case, the sensors will be detected while the user is moving, causing strong noise and movement artifacts. A strong peak detection algorithm is needed so that we only send valid data. The R-peak detection/filtering algorithm implemented in Figure 3.9 has 5 stages to ensure that the board filters out invalid signals. First while detecting the analog ECG values a 3-sample moving average is done on the values. Secondly, once the ECG values have been received a peak is only detected when the analog signal exceeds over 70% of its max analog-to-digital converter (ADC) value. Third, peaks are rejected if they are within 60ms of each other to prevent double counting. The fourth step happens when a buffer window full of these R-peaks, where only RR values (Time between successive R-peaks) are between 300ms-1500ms, if not, those peaks are dropped. The last stage of the filtering algorithm is that if an RR interval is beyond +/- 400ms of the most recent 10s window, if they fail in this condition those peaks are also dropped.

During development, to verify the correctness of the R-peak detection algorithm developed, it was compared against a set of ground truth ECG signal peaks. An ECG signal with simulated noise and strong motion artifacts and its associated peaks were first generated and retrieved. Afterwards, all the peak values simulated were compared against two conditions specified in peer reviewed medical journals. If valid they were considered peaks validated by medical ground truths. These conditions where every RR interval must be within 300ms to 1300ms [19] and that if an abrupt change of RR interval distance of >20% was detected from the mean of the past 5 intervals, it should be dropped [20]. Afterwards, we applied the R-peak detection algorithm in comparison to the peaks validated by the ground truth results and found the algorithm got a recall of 82%. Highlighting that the algorithm under strong artifact and noise is still within 80% of accurately detecting peaks. The first 20 seconds of this simulation is shown in Figure 3.11.

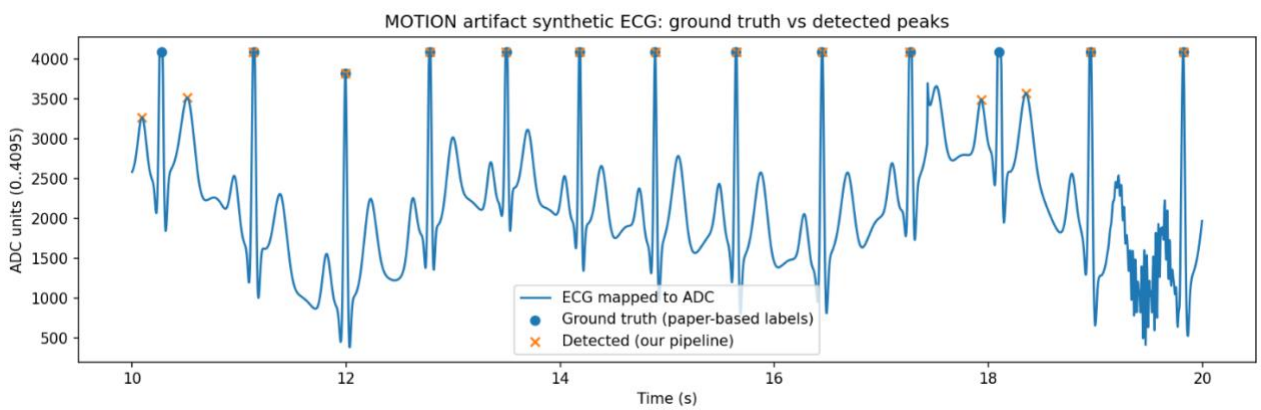

Figure 3.11 Motion Artifact Synthetic ECG: ground truth vs detected peaks graph

During the design and planning process for the microcontroller system, several iterations were considered before arriving finalizing the hardware. Initially, we planned to use Bluetooth for wireless communication due to its simplicity in setup. The options explored included an Arduino Nano paired with a Bluetooth module, or the ESP-WROOM-32 board, which offers both Bluetooth and Wi-Fi functionality.

Both options satisfy FS6 and FS7, as they have sufficient analog pins capable of performing analog-to-digital conversions (ADC) on the ECG signal, and their microprocessors can execute code developed in standard integrated development environments (IDEs) for signal processing and data

transmission. Both boards can also operate at configurable sampling frequencies. Thus, if I sample at 250Hz with a window size of data for every 1.5 seconds as shown in Figure 3.8, sending data via Bluetooth is well under 6.5 seconds on average. Since both boards' microprocessors can implement these windows and sampling patterns, both options pass NFS7.

However, Bluetooth presents challenges with range, impacting NFS8. While BLE can operate in long-range mode (exceeding 200m), two problems arise. Firstly, bandwidth drops down to 100-500kbps (compared to 1–3 Mbps for standard short-range Bluetooth). Secondly, iOS devices do not enforce long-range Bluetooth by default. Meeting NFS8 would therefore require an ad hoc system with several StaminaSense devices running and the web application always near one of them. Thus, with this complication we investigated using Wi-fi to satisfy NFS8 with a less complex solution. Additionally, Wi-fi in the ESP32 can achieve theoretically up to 150mbps, far exceeding the bandwidth requirement, to work with sensors satisfying NFS11, the bandwidth requirement.

To address this, we investigated using Wi-Fi to satisfy NFS8 with a less complex solution. The ESP32's Wi-Fi can reach up to 150 Mbps, easily exceeding the bandwidth requirements for sensor data transmission (NFS11).

The final iteration of our design uses the ESP-WROOM-32 board, as it still satisfies FS6, FS7 and NFS7 while leveraging Wi-Fi instead of Bluetooth. By using Wi-Fi, we implemented a non-connection-based transmission system. The ESP32 hosts a web server, allowing the mobile application to access and parse data as needed. Each time the transmission window is filled, the microcontroller sends the data as a JSON payload over HTTP to the web server, maintaining NFS7. When users are on the same Wi-Fi network, the 200m range requirement for NFS8 is easily met.

After comparing both boards, using the decision matrix in Table 3.4, the ESP-WROOM-32 was chosen for its Wi-Fi capabilities and superior ADC components. The ESP32's ADC provides 12-bit resolution, compared to the Arduino Nano's 10-bit ADC, enabling more precise quantification of analog ECG signals.

Table 3.6 Microcontroller Decision Matrix

Criteria	Weight	**ESP 32-Wifi**	**ESP 32- BLE**	**Arduino Nano+BLE**
Ease of Use (IDE)	0.1	0.8	0.8	1
Communication (Range/Speed/Ease)	0.5	1	0.2	0.2
ADC Port support	0.4	1	1	0.3
Total		0.98	0.58	0.32
Rank		1	2	3

Quantitative Analysis of why more ADC resolution bits are better:

The ESP32's ADC is 12-bit with a 3.3 V V_{in}, giving:

$$ADC\ Resolution\ Esp = \frac{3.3v}{2^{12}} = \frac{3.3v}{4096} \approx \mathbf{0.}\frac{\mathbf{806mV}}{\mathbf{LSB}}$$

By comparison, an Arduino Nano's 10-bit ADC gives (Powered by 5v V_{in}):

$$ADC\ Resolution\ Uno = \frac{5.0v}{2^{10}} = \frac{5.0v}{1024} \approx \mathbf{4.}\frac{\mathbf{88mV}}{\mathbf{LSB}}$$

The ECG module described in section 3.2.2 amplifies the raw ECG signal from 1mv (with peaks of 5mv) [14] to a readable range of roughly 0.5 V-2.5 V. With this signal range, both the ESP32 and Arduino boards can resolve the ECG signal using their ADCs. However, the step size of the ADC differs significantly between the two boards. The ESP32 has a step size of 0.806 millivolts per least significant bit (mV/LSB), compared to 4.88mV/LSB for the Arduino Nano, giving the ESP32 approximately 6.05 times finer resolution ($4.88 \div 0.806 \approx 6.05$). This increased resolution allows the ESP32 to digitize the ECG signal with much greater precision, which is why it achieved a performance rating of 1, compared the Nano's 0.3. The Arduino Nano does work, but with more degraded precision.

3.2.2 Hardware/Sensors

The StaminaSense hardware and sensor subsystem consists of a SparkFun Heart Rate Monitor v1.0 [18], which is based on the AD8232 chip and uses a 3.5 mm headphone jack interface to receive ECG signals. The system also incorporates a CAB-12970 [11], which connects to the three H124SG [10] placed on the user's body.

Using these components, the AD8232 evaluation board acquires the ECG signal, performs noise filtering, and amplifies the signal to a usable voltage range. The resulting output is then transmitted to the MCU (ESP32 Microcontroller) via pin connections, as defined in Table 3.3.

The hardware system is designed to be worn using a compression vest. The gel electrodes will make direct contact with the user's skin underneath the vest. The vest ensures the electrodes are firmly held in place without requiring frequent reattachment. The electrodes are connected via a 3-to-1 cable, which terminates in a 3.5 mm audio jack that plugs into the ECG evaluation board. The electrode pads and cables are lightweight and minimally intrusive, eliminating the need for additional housing.

In contrast, the microcontroller (ESP32), ECG evaluation board, and power supply are enclosed within a compact 3D-printed size case to securely house all components. The 3D-printed enclosure can be seen in Figure 3.12. This enclosure is placed inside of a small bag, made for camera lenses, and is strapped near the user's waist with a belt attached to the pouch which can be seen in Figure 3.13.

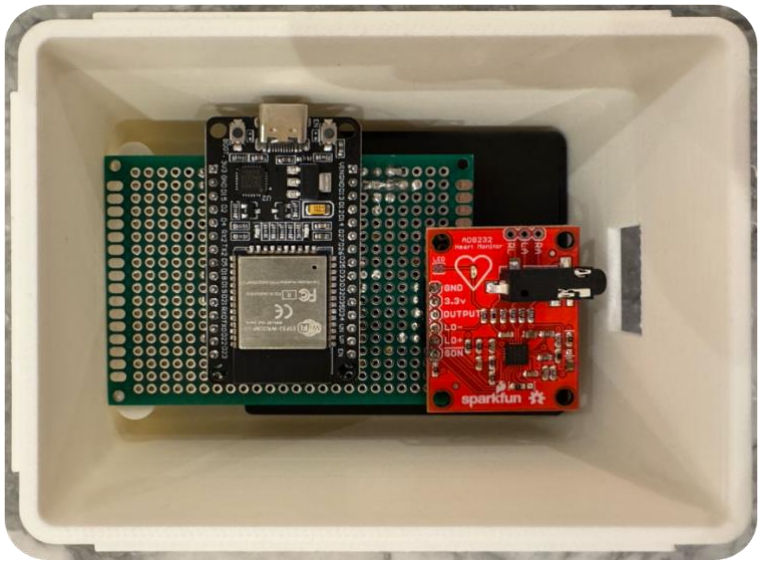

Figure 3.12 3D-Printed Enclosure

Figure 3.13 Wearable Prototype

The gel electrodes are positioned on opposite sides of the heart, with a third electrode placed on the lower right side of the torso. The electrode configuration can be followed from the layout illustrated in Figure 3.14.

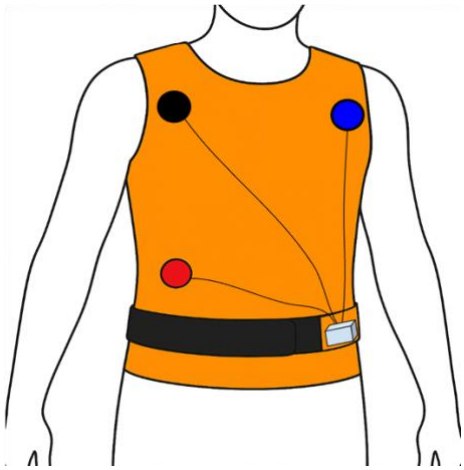

Figure 3.14 Electrode Configuration

These design choices support NFS9 and NFS10 by reducing motion artifacts and the likelihood of a disconnection. Placing the hardware near the waist lowers the probability of accidental impact from external objects, while the compression vest ensures consistent electrode contact, reducing signal noise caused by movement. Combined with the filtering capabilities of the ECG module, this improves overall signal accuracy.

Additionally, potential risks to the electronics, primarily sweat exposure and mechanical shock, are mitigated through the placement and enclosure. The waist location minimizes exposure to high-impact forces, while the 3D-printed case and pouch provide a protective barrier against sweat, supporting NFS4. Throughout the testing conducted, this system has been able to maintain reliable operation for at least 100 hours.

Through quantitative analysis, we demonstrate why AD8232 is perfect for our use case.

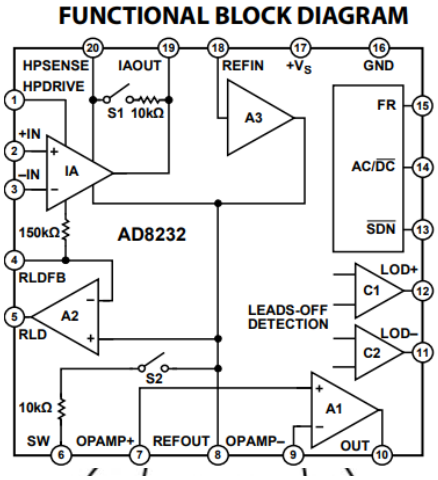

Figure 3.15 AD8232 Functional Block Diagram

Table 3.7 Functional Block Diagram: Values for HPF come from the eval board [18]

Block	Pins	Function	Equation	Purpose
IA (Instrumentation Amp)	2,3,19	Amplifies differential ECG signals. 150kΩ sets input impedance. High CMRR suppresses 60Hz noise (Power line frequency).	$V_{out} = G_{IA} \times (V_+ - V_-)$, G_{IA} = The Op Amp Gain	Rejects common-mode signals
High-Pass Filter	1,20	Forms high-pass filter with external cap.	$f_c = \frac{1}{2\pi RC}$, Capacitance is set on eval board: With R11 = 10MΩ and C9 = 0.1μF [18]: $f_c \approx 0.16$ Hz	Eliminate low-frequency elements
RLD (Right-Leg Drive)	4,5	Feeds back inverted common-mode voltage for noise cancellation.		Actively cancels common-mode signals before they reach the IA
OPAMP/REFOUT	6,7,8,9	Pin 8 provides mid-supply reference (1.65V). Buffers and sets reference voltage.	$V_{ref} = \frac{3.3}{2} = 1.65V$	Center output signal, prevent clipping
Output	10,11,14	Pin 10: main output. Pins 11,12: lead-off detection. Pin 14: AC/DC mode.	$V_{ref} = V_{ref} \pm Amplified\ ECG$	Clean, filtered ECG ready for ADC
Power/Reference	16,17,18	Pin 17: +Vs supply. Pin 16: GND. Pin 18: custom reference input.	V_{supply} = 3.3V	Power and voltage reference

The functional block diagram shown in Figure 3.15 is explained in Table 3.5. Based on the analysis of the AD8232's functionality, using an evaluation board in conjunction with the integrated circuit enables a clean ECG signal acquisition from gel electrodes through a 3-lead input configuration. The instrumentation amplifier (pins 2 and 3) captures and amplifies the weak differential heart signals while rejecting common-mode noise. A high-pass filter (pins 1 and 20), with a cutoff frequency of approximately 0.16 Hz, removes baseline and motion noise.

Additionally, the right-leg drive (RLD) circuit (pins 4 and 5) actively suppresses the power-line interference (e.g., 60 Hz noise) by feeding an inverted common-mode signal back into the body. The internal operational amplifiers and reference system (pins 6-9) establish a stable 1.65 V reference point, centering the ECG waveform and preventing signal clipping. The final output stage (pins 10-12 and 14) produces a clean, amplified ECG waveform with lead-off detection capability. The power and reference pins (16-18) ensure stable 3.3 V operation with optional custom reference input.

Together, these subsystems generate a filtered and amplified ECG signal suitable for ADC conversion and direct visualization. The integrated filtering mechanisms help mitigate motion-induced noise, supporting NFS9 and NFS10. To further improve robustness, an additional software-based filtering step is implemented on the microcontroller to remove outliers and reduce the impact of transient motion artifacts.

This subsystem involved very limited design alternatives, as the AD8232 is one of the most widely used and commercial ECG integrated circuits. The primary design decision was between using a prebuilt evaluation board, implementing the AD8232 IC directly, or designing a custom PCB. Given project time constraints, we opted to go with the prebuilt board.

3.2.3 Power Supply

The StaminaSense power supply consists of a battery pack using for AA rechargeable nickel-metal hydride (NiMH) batteries connected in series, along with an integrated on/off switch. This configuration satisfies FS8, as the system is fully rechargeable. Each NiMH AA battery provides approximately 1.2 V [16], resulting in a total output of 4.8 V, which meets NFS5 by supplying an appropriate voltage range for the ESP32 microcontroller (3.3-6 V).

The selected batteries are Amazon Basics Rechargeable AA Batteries, each with a capacity of 2000 mAh. NiMH AA batteries typically offer capacities ranging from 600 mAh to 2750 mAh [16], ensuring sufficient energy storage for extended operation of the system.

The power supply is connected to the V_{in} pin of the microcontroller, as described in Section 3.2.1. This configuration leverages the onboard linear voltage regulator, which steps the input voltage down to a stable 3.3 V required by the ESP32. The excess voltage (4.8 V – 3.3 V = 1.5 V) is dissipated as heat through the regulator, which is within acceptable operating conditions.

Using four AA batteries is preferred over three, as a three-cell configuration would provide approximately 3.6 V, which is too close to the minimum required voltage and may become insufficient if the batteries are partially discharged. In contrast, a four-cell configuration provides a more reliable voltage margin, ensuring stable operation even as battery levels decrease. Therefore, four NiMH AA batteries offer a more reliable power solution for the system.

The battery case also provides an on/off switch allowing the user to easily power the system without requiring software-based sleep functionality or physically disconnecting the power supply.

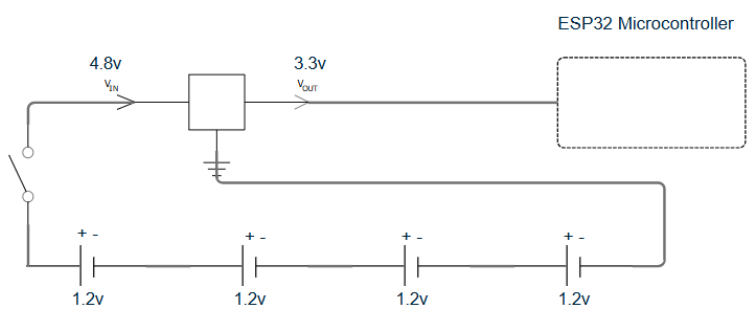

Figure 3.16 Full Schematic with ESP32 without Voltage Regulation on the MCU

Each AA NiMH battery we are using in this power supply has v = 1.2 V and Capacity = 2000 mAh. Quantitatively analyzing this circuit, shown in Figure 3.16, since these batteries are in <u>series</u>:

$$V_{battery} = \sum v_i = 1.2 * 4 = \mathbf{4.8v}$$

Capacity (mAh) remains constant in a series circuit. Thus if:

$$C_{battery} = C_i = \mathbf{2000mAh}$$

The equivalent Schematic/circuit is shown in Figure 3.17 below.

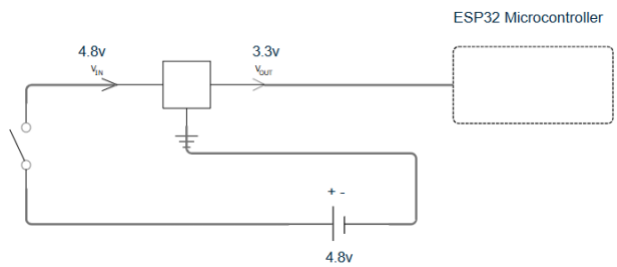

Figure 3.17 Reduced Schematic with ESP32

Since the series voltage of this power supply built with a linear battery circuit provides an output voltage of 4.8 V this satisfies NFS5, since this is a safe range for the voltage regulator to convert 4.8 V to 3.3 V.

According to [17], the ESP32 microcontroller draws between 160–260 mA in its active state, with an average current consumption of 180 mA. In comparison, the AD8232 ECG module draws a negligible current in the microamp range when connected to the MCU. Thus, the total system current is dominated by the ESP32. Based on this, a quantitative analysis of the power supply runtime at full charge can be performed as follows:

ESP32 MCU power draw (V = 3.3 V post linear regulation):

$$P = VI = 3.3\,V \times 0.18\,A = 0.594\,W \text{ of operational power consumption}$$

Power supply total energy in watt hours:

$$W \times h = C \times V = 2\,Ah \times 4.8\,V = 9.6\,Wh$$

Active system continuous runtime:

$$Runtime = \frac{Energy(Wh)}{Opterational\ Consumtion} = \frac{9.6Wh}{0.594W} = \mathbf{16.16\ hours}$$

Thus, the power supply subsystem can support continuous operation for approximately 16 hours, accounting for the power consumption of the ESP32 microcontroller and the ECG module. Through testing, we have also verified the battery life through testing done on the prototype. This meets the NFS6 requirement for minimum operational time before recharging is necessary.

Before selecting four AA rechargeable batteries, we considered using a single 3.7 V lithium-ion battery. This option provides the required voltage, satisfying NFS5, and has a capacity approximately half that of the 4 AA batteries. Despite the lower capacity, the lithium-ion battery would still support around 8 hours of operation, exceeding the NFS6 requirement of 2 hours. Lastly, lithium-ion batteries are rechargeable satisfying FS8. Therefore, all system specifications would also be met using this alternative,

demonstrating that the decision to select four AA batteries was made through a fair and competitive evaluation of viable options with a decision matrix seen in Table 3.6.

Table 3.8 Power Supply Decision Matrix

Criteria	Weight	**4xAA NiMH**	**3.7 V Lipo**
Safety	0.7	1	0.5
Weight	0.3	0.3	1
Total		0.79	0.65
Rank		1	2

After comparing the different options, as described in the power supply subsystem, we selected four AA NiMH rechargeable batteries.

The rating rationale is as follows: AA batteries are widely used, well-tested and have minimal safety concerns. In contrast, lithium-ion batteries have several safety warnings and pose a risk of combusting, particularly in warm environments. Since athletes generate heat while running or exercising, lithium-ion batteries would require additional safety measures, such as specialized enclosures, airflow management, and insulation. AA batteries, by comparison, are inherently safer and can be house in a simple series battery case without extensive precautions. Furthermore, course safety requirements mandate that high-capacity lithium-ion batteries have well-insulated connections and easy disconnect mechanisms, adding complexity to the implementation.

In terms of weight, four AA batteries with a case weigh approximately 190 g, significantly heavier than a small 3.7 V lithium-ion cell (5-25 g). Despite this, the battery pack remains the heaviest component of the wearable system, as the MCU, ECG module, and cables together weigh less than 0.5 lbs. The compression vest weighs approximately 150 g, the total weight of the system remains below 10 lbs, satisfying NFS3.

Overall, the AA battery choice provides a safe, reliable and practical solution while maintaining an acceptable total system weight.

3.3 External Inputs/Outputs

This subsystem contains three separate external objects. The mobile device the StaminaSense app is running on, a user observing the interface, and a user wearing the hardware connecting the sensors to the mobile application.

The mobile device can be any platform that satisfies the following two requirements: it must be capable of running mobile applications on either iOS or Android, and it must have access to the internet via Wi-Fi or cellular data. This is necessary because, as shown in Figure 3.8, the ESP32 microcontroller hosts a web server that streams the user's ECG data. The mobile device must be able to connect to this web server to fetch and display the data. Therefore, any device that meets these criteria and can install the StaminaSense application is compatible with the system.

4 Discussion and Conclusions

4.1 Evaluation of Final Design

Overall, the final StaminaSense design effectively meets the project objective of providing a comprehensive overview of an athlete's physiological state through a wearable system integrated with machine learning. The compression vest, combined with ECG sensing hardware and the ESP32 processing unit, successfully enables real-time data collection, preprocessing, and wireless transmission to a mobile interface. This supports accurate heart rate variability analysis, which can be used by our machine learning model to predict a stamina level, helping coaches assess injury risk. Additionally, the system satisfies all functional and non-functional specifications, including power efficiency, signal accuracy, wireless range, safety, and usability. As a result, the final design provides a reliable and practical solution that allows coaches to make informed training and recovery decisions, leading to improved athletic performance and reduced injury risk.

4.2 Use of Advanced Knowledge

This project design leveraged several concepts from third-year ECE courses at the University of Waterloo.

Signal processing knowledge from ECE318 (Signals and Systems) was used extensively in the microcontroller subsystem, specifically in sampling and analyzing the ECG signals to extract HRV metrics. The chosen sampling frequency of 250 Hz was justified using the Nyquist Theorem, ensuring accurate representation of the QRS complex in ECG data. Additionally, ADC resolution bit analysis guided the hardware selection by quantitatively comparing the precision of analog-to-digital conversion, impacting the HRV measurement accuracy.

For the backend subsystem, concepts from ECE356 (Database Systems) were applied to design a normalized PostgreSQL schema that stores static and dynamic biometric data efficiently. A space-saving strategy reduced storage costs by 97.5%, demonstrating a clear application of database normalization and query optimization.

In the machine learning subsystem, techniques from ECE457B (Fundamentals of Computational Intelligence) informed the selection of the Extra Trees Model. The application of input normalization and the theoretical evaluation of model options (KNN vs. GMM) reflect an understanding of clustering algorithms and model generalization that powered development in the early iterations. Another important concept that helped inform our final decision on which model to use is ensemble learning, where multiple weak learners are combined to produce a strong overall classifier. As the models are compared across multiple iterations, ensemble-based approaches such as Extra Trees, Random Forest, and Gradient Boosting consistently produce the strongest results. This reinforces the course concept that aggregating many predictions into one averaged final prediction can improve generalization compared to relying on a single simpler model which we saw when observing GMM against Extra Trees. One final important

concept covered in 457B that was crucial in the evaluation of our models was cross-validation. By using 10-fold stratified cross-validation, we were able to evaluate each candidate model on a more reliable basis and as a result we were able to select the model that performed well across multiple splits as opposed to just one.

Altogether, these components show an application of fourth-year ECE knowledge across signal processing, database systems, and machine learning.

4.3 Creativity, Novelty, Elegance

The StaminaSense project's innovation is found not in a single component, but in the elegant synergy between its hardware, data architecture, and machine learning model to solve a novel problem.

The core novelty of the project lies in its approach to assessing athlete fatigue. Instead of relying on generic, one-size-fits-all models, we employ an Extra Trees Model that demonstrates strong generalization across athletes with varying physiological profiles. Before each monitoring session, the system establishes a physiological baseline for the athlete. All subsequent biometric inputs (e.g., Heart Rate Variability from the ECG sensor) are then normalized as a percent change relative to that baseline before being passed to the model. This approach ensures that fatigue predictions are made relative to an individual's own resting state, making the output more personalized and more relevant to the user’s actual condition.

The project's elegance is demonstrated by the seamless integration between the database design and the machine learning model's requirements. To support per-athlete normalization, the database creatively stores a baseline biometric reading captured while the athlete is in a rested state. Subsequent sensor readings are interpreted relative to this stored baseline, allowing the backend to prepare model inputs in a consistent and efficient way. Rather than storing every raw measurement, keeping track of only a single baseline reading reduces storage costs significantly (by an estimated 97.5%). This design is a clear example of an elegant, purposeful architecture where each component is designed to efficiently serve the needs of the others.

While fitness wearables are common, the StaminaSense system is creative in its specific application and integrated design. It addresses the high-value and niche problem of preventing athlete over-exhaustion and injury which is a domain not adequately served by consumer-grade devices. The integration of a medical-grade ECG sensor into a non-intrusive compression vest, which wirelessly communicates with a sophisticated, cloud-based personalization engine, represents a creative and holistic system designed from the ground up to provide actionable, data-driven recommendations to coaches and athletes.

4.4 Quality of Risk Assessment

Risks were effectively mitigated in 498A by initiating prototype development early to identify potential technical hurdles. Although initial planning suggested that time management was the only significant constraint, active work revealed two primary risks: data collection and wearable device construction. We determined that the machine learning model required a larger dataset than initially anticipated for accurate operation. Furthermore, our limited experience in clothing design and 3D printing

necessitated allocating more time for integrating sensor pads, wiring the vest, and creating a durable 3D-printed housing for our electronics. By uncovering these challenges and risks early, we successfully developed a mitigation strategy to address these blockers.

We strategically mitigated these risks by reallocating our project timeline across the 498A and 498B terms. Recognizing that data collection was unfeasible within a single semester, we decoupled this task from the original development phase. In 498A, we prioritized core prototype functionality, completing 75% of the technical requirements while deferring data gathering to avoid the inefficiencies of doing too many tasks at once. This allowed us to utilize the co-op term between 498A and 498B for data collection and labelling. Therefore, we were able to enter 498B with a complete dataset, leaving only the physical integration of the wearable device. This approach minimized parallel task management and ensured that high-effort tasks received strong focus during optimal time periods.

A significant hurdle came from the limited availability of hardware, as we only had two working prototypes shared amongst four members. Two team members were also located in different US states during the work term, which further complicated things. To optimize our timeline, the Canadian-based members conducted data collection before transferring the devices to the remaining members during holiday breaks. This handover allowed the US team to begin developing the machine learning model with direct hardware access for testing. Our geographic proximity during the holidays facilitated an easy rotation, ensuring we could maintain a continuous workflow and achieve a diverse training set.

4.5 Student Workload

The group work has involved collecting data, training the machine learning model, setting up the backend server, designing the database schema, ordering and setting up the hardware, receiving ethics approval, creating the mobile application, creating the poster, and assembling the wearable prototype. Table 4.1 displays the total number of hours each group member has contributed to this project.

Table 4.1 Student Workload

Group Member	*ID*	*Percentage (%)*
Jinha Kim	20936456	25
Michael Tham	20945854	25
Kordian Mazurkiewicz	20933336	25
Eric Chanthalima	20960497	25

5 References

[1] UEFA, "New format for Champions League post-2024: Everything you need to know | UEFA Champions League," *UEFA.com*, Oct. 23, 2023. https://www.uefa.com/uefachampionsleague/news/0268-12157d69ce2d-9f011c70f6fa-1000--new-format-for-champions-league-post-2024-everything-you-ne/

[2] P. MacInnes, "Injuries cost top men's clubs in Europe €732m last season, study shows," *the Guardian*, Oct. 15, 2024. https://www.theguardian.com/football/2024/oct/15/injuries-cost-top-men-clubs-europe-record-732m-last-season-study-shows

[3] "Catapult One Team | Soccer," *Catapult One*. https://one.catapultsports.com/us/catapult-one-team-soccer/

[4] C. J. Holmes, H. V. MacDonald, M. R. Esco, M. V. Fedewa, S. A. Wind, and L. J. Winchester, "Comparison of heart rate variability responses to varying resistance exercise volume-loads," *Research Quarterly for Exercise and Sport*, vol. 93, no. 2, pp. 391–400, 2022. doi: 10.1080/02701367.2020.1851351.

[5] F. Sabry, T. Eltaras, W. Labda, K. Alzoubi, and Q. Malluhi, "Machine Learning for Healthcare Wearable Devices: The big picture," Journal of healthcare engineering, https://pmc.ncbi.nlm.nih.gov/articles/PMC9038375/

[6] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

[7] K. P. Murphy, Machine Learning: A Probabilistic Perspective. Cambridge, MA, USA: MIT Press, 2012.

[8] Espressif Systems. ESP32-WROOM-32: Wi-Fi & Bluetooth Module Datasheet (Version 3.5). 2023. https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf

[9] Analog Devices. AD8232 Heart Rate Monitor – Single-Lead, Heart Rate Monitor Front End. Compact ECG IC designed for wearable heart monitoring and signal conditioning. https://www.analog.com/media/en/technical-documentation/data-sheets/ad8232.pdf

[10] Covidien. H124SG ECG Electrodes – Product Specification Sheet. SparkFun Electronics, hosted copy. Pre-gelled Ag/AgCl snap electrodes designed for surface biopotential measurements (ECG/EMG), with foam backing and hydrogel adhesive. https://cdn.sparkfun.com/datasheets/Sensors/Biometric/H124SG.pdf

[11] SparkFun Electronics. CAB-12970: Electrode Cable for Biomedical Pads – Datasheet. Three-conductor lead wire cable with snap connectors for biomedical sensors, commonly used with the AD8232 ECG board. https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/2277/CAB-12970_Web.pdf

[12] Last Minute Engineers. ESP32 Pinout Reference: Which GPIO Pins Should You Use? December 21, 2020. Comprehensive breakdown of ESP32 GPIO functionality, including ADC, DAC, PWM, SPI, I²C, UART, touch sensors, and strapping pin behavior. https://lastminuteengineers.com/esp32-pinout-reference/

[13] M. Shaffer and J. P. Ginsberg, "An Overview of Heart Rate Variability Metrics and Norms," Frontiers in Public Health, vol. 5, p. 258, Nov. 2017. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4624499/

[14] A. N. Sassi and M. G. Cerutti, "Advances in heart rate variability signal analysis: joint position statement by the e-Cardiology ESC Working Group and the European Heart Rhythm Association co-endorsed by the Asia Pacific Heart Rhythm Society," European Heart Journal - Digital Health, vol. 1, no. 1, pp. 3–6, Nov. 2020. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7664289/

[15] *Electrocardiography*, Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Electrocardiography. [Accessed: Jun. 23, 2025].

[16] AA battery, Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/AA_battery

[17] *ESP32 Sleep Modes & Their Power Consumption*, Last Minute Engineers. [Online]. Available: https://lastminuteengineers.com/esp32-sleep-modes-power-consumption/

[18] SparkFun Eval Board for AD8232_Heart_Rate_Monitor Eval board schematic https://cdn.sparkfun.com/datasheets/Sensors/Biometric/AD8232_Heart_Rate_Monitor_v10.pdf

[19] Vest, Adriana N *et al.*, "An open source benchmarked toolbox for cardiovascular waveform and interval analysis," *Physiological Measurement*, vol. 39, no. 10, p. 105004, Oct. 2018, doi: 10.1088/1361-6579/aae021.

[20] M. Benchekroun *et al.*, "Preprocessing methods for ambulatory HRV analysis based on HRV distribution, variability and characteristics (DVC)," *Sensors*, vol. 22, no. 5, p. 1984, Mar. 2022, doi: 10.3390/s22051984.